PHANTOM

Autonomous AI Penetration Testing Agent

Technical Report

Version 0.8.5 — February 2026

Author: Gadouri Redwan

Repository: `github.com/Usta0x001/Phantom`

Package: `pip install phantom-agent`

Sandbox: `ghcr.io/usta0x001/phantom-sandbox:latest`

License: Apache 2.0

Complete technical description of the Phantom system: architecture, components, operational flow, execution layers, and verified test results.

Abstract

Phantom is an autonomous AI-powered penetration testing agent that performs full web application security assessments without human intervention. Given a target URL and an LLM API key, Phantom spawns a tree of specialized agents, each capable of reasoning about the target, selecting and executing security tools, interpreting results, and escalating findings. All tool execution occurs inside an isolated Docker sandbox containing more than 20 professional security tools. Phantom produces structured reports in HTML, Markdown, JSON, SARIF, and PDF formats, with full MITRE ATT&CK and OWASP Top 10 mapping. The current release (v0.8.5) ships with 146 passing tests, is published on PyPI, and supports any LLM provider via LiteLLM.

Contents

1 System Overview **3**
- 1.1 High-Level Operational Flow . . . 3
- 1.2 Key Design Principles . . . 3

2 System Architecture **4**
- 2.1 Package Structure . . . 4
- 2.2 Component Responsibilities . . . 5

3 Agent Framework **5**
- 3.1 Agent Types . . . 5
- 3.2 Agent Tree Architecture . . . 6
- 3.3 Agent Lifecycle . . . 6
- 3.4 Context & Memory Management . . . 6

4 Sandboxed Execution Environment **6**
- 4.1 Architecture . . . 7
- 4.2 Tool Server API . . . 7
- 4.3 Security Isolation . . . 7
- 4.4 Pre-installed Security Tools . . . 8

5 LLM Integration Layer **8**
- 5.1 Provider-Agnostic Design . . . 8
- 5.2 Configuration . . . 9
- 5.3 Supported Model Identifiers . . . 10
- 5.4 Retry & Fallback Chain . . . 10

6 Scan Workflow **10**
- 6.1 Scan Modes . . . 10
- 6.2 Scan Pipeline . . . 11
- 6.3 Differential Scanning . . . 11

7 Detailed System Execution Plan **11**
- 7.1 Layer Stack . . . 11
- 7.2 Step-by-Step Execution Trace . . . 11
  - 7.2.1 Step 1 — CLI Invocation (L0) . . . 12
  - 7.2.2 Step 2 — Configuration Resolution (L1) . . . 12
  - 7.2.3 Step 3 — Sandbox Startup (L2) . . . 12
  - 7.2.4 Step 4 — Root Agent Initialization (L3) . . . 12
  - 7.2.5 Step 5 — Reconnaissance Phase (L3 + L4 + L5 + L6) . . . 13

7.2.6 Step 6 — Vulnerability Analysis Phase (L3 + L4) 13
7.2.7 Step 7 — Exploitation Phase (L3 + L4 + L5 + L6) 13
7.2.8 Step 8 — Validation Phase (L3 + L4) 14
7.2.9 Step 9 — Report Generation (L8) 14
7.2.10 Step 10 — Sandbox Teardown (L2) 14
7.3 Full Data Flow Diagram 15

8 Knowledge Representation 15
8.1 Knowledge Graph 15
8.2 Vulnerability Data Model 16

9 User Interface 16
9.1 Command-Line Interface 16
9.2 Text User Interface (TUI) 17
9.3 CI/CD Integration 17

10 Reporting System 18

11 Installation & Deployment 18
11.1 Prerequisites 18
11.2 Installation 18
11.3 Published Images 19

12 Testing & Verification 19
12.1 Test Suite 19
12.2 Sandbox Verification (February 2026) 20

13 Security & Ethical Considerations 20
13.1 Authorization Enforcement 20
13.2 Scope Validation 20
13.3 Data Privacy 20
13.4 Responsible Use 21

14 Project Status & Publication 21

15 Conclusion 21

A Quick Reference 22

B Environment Variables Reference 23

1. System Overview

Phantom is a complete, self-contained penetration testing platform consisting of two parts:

1. **Phantom CLI** — the Python package installed by the user (`pip install phantom-agent`). Handles configuration, LLM communication, agent orchestration, and reporting.
2. **Phantom Sandbox Image** — a Docker image (`ghcr.io/usta0x001/phantom-sandbox:latest`) containing the tool server and all security tools. Pulled and launched automatically per scan.

The two components communicate over an authenticated HTTP channel. The user never manages the sandbox directly.

1.1 High-Level Operational Flow

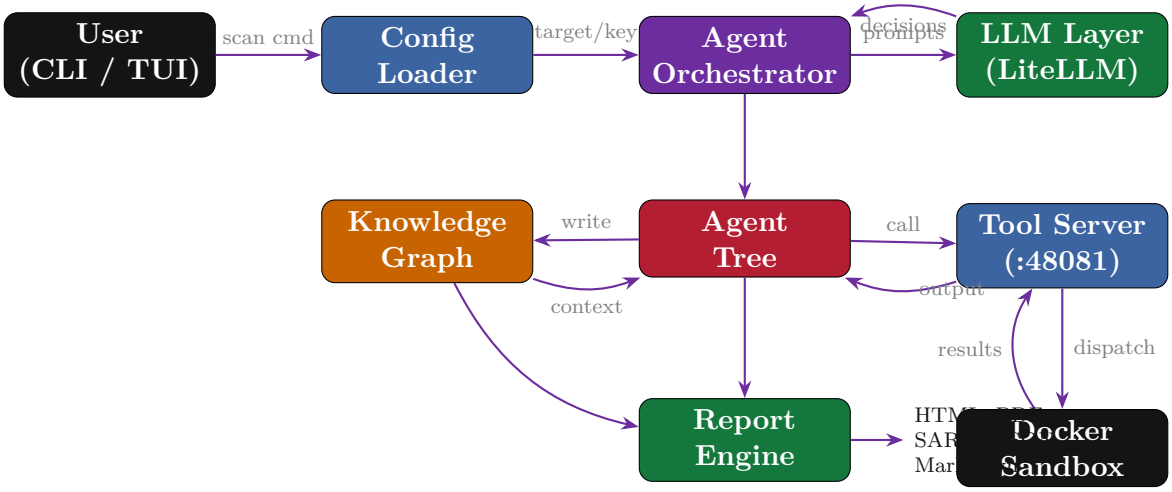

Figure 1: High-level operational flow of the Phantom system

1.2 Key Design Principles

Zero human-in-the-loop Once launched, Phantom runs to completion without asking for input.

Proof-of-concept first Every reported vulnerability is accompanied by a working exploit demonstration.

Portable by design The CLI installs via `pip`; the sandbox image is pulled automatically. No manual setup required.

Provider-agnostic Any LLM supported by LiteLLM (OpenAI, Anthropic, Google, Groq, Ollama, ...) works out of the box.

Sandboxed execution All tool invocations run inside an isolated Docker container. The host system is never touched.

2. System Architecture

2.1 Package Structure

Listing 1: Phantom package directory tree

```
phantom/
 agents/        # Agent state machines, protocols, EnhancedAgentState
 config/        # PhantomConfig (Pydantic v2); persistent JSON settings
 interface/     # cli_app.py (Typer CLI), tui.py (Textual TUI)
 models/        # Pydantic models: Vulnerability, ScanResult, KnowledgeGraph
 runtime/
   docker_runtime.py  # Sandbox lifecycle: start, health-check, stop
   tool_server.py     # FastAPI server running INSIDE the sandbox
   runtime.py         # Abstract runtime interface
 skills/        # Reasoning skill modules (.md knowledge bases)
 telemetry/     # Logging, tracing, metrics
 tools/         # Tool definitions: terminal, browser, file, proxy
 utils/         # SARIF formatter, diff engine, scope validator
 workflows/     # High-level scan workflows: quick, standard, deep
```

2.2 Component Responsibilities

Table 1: Core Component Responsibilities

Component	Responsibility
`PhantomConfig`	Stores user settings in `~/.phantom/config.json`. Pydantic v2 with full validation.
`cli_app.py`	Typer CLI: `phantom scan`, `phantom config`, `phantom report`.
`docker_runtime.py`	Pulls sandbox image if absent; spawns container; polls `/health`; manages teardown.
`tool_server.py`	FastAPI server inside sandbox. Exposes `POST /execute` and `GET /health`.
`AgentOrchestrator`	Spawns root agent; manages tree; mediates LLM calls; enforces scope; handles retries.
`KnowledgeGraph`	Shared in-memory graph updated in real time. Provides context injection to all agents.
`ReportEngine`	Aggregates findings; renders HTML/Markdown/SARIF/JSON/PDF; maps to MITRE/OWASP.

3. Agent Framework

3.1 Agent Types

Table 2: Phantom Agent Types

Agent	Role	Primary Responsibilities
Root / Orchestrator	Coordinator	Creates plan, spawns specialist agents, aggregates results.
Reconnaissance	Discovery	Port scanning, service fingerprinting, tech detection, dir brute-forcing.
Vulnerability Analysis	Analysis	Identifies attack surface, prioritizes candidates, assigns CVSS estimates.
Exploitation	Attack	Tests candidates with sqlmap, nuclei, custom payloads. Generates PoC.
Post-Exploitation	Depth	Explores impact: data access, privilege, lateral movement.
Validation	Verify	Confirms true positives; eliminates false positives; records evidence.
Reporting	Output	Structures findings, assigns severity, maps to OWASP/MITRE.

3.2 Agent Tree Architecture

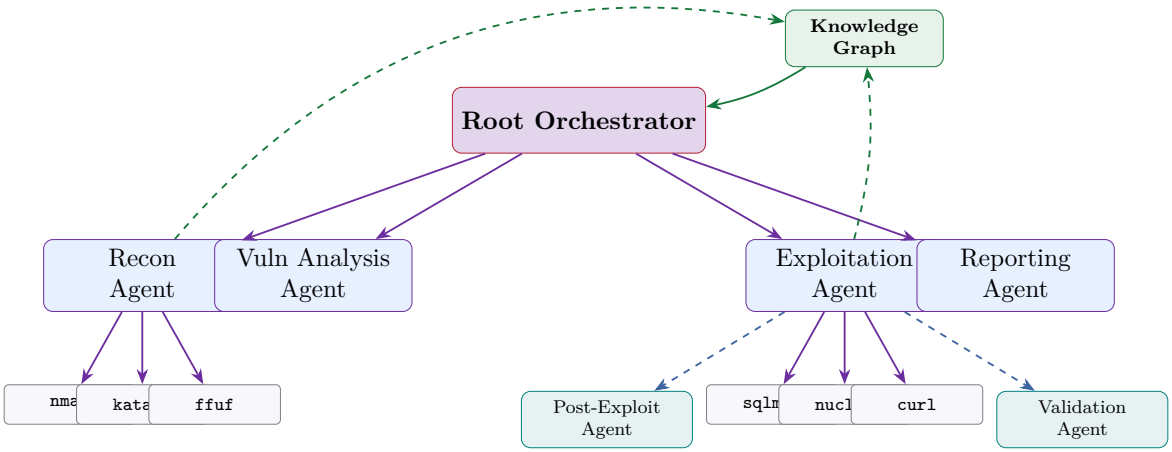

Figure 2: Agent tree: solid arrows = spawn/command, dashed blue = sub-agent spawn, dashed green = KG updates

3.3 Agent Lifecycle

Each agent follows this deterministic state machine:

1. **INIT** — Receive system prompt + current KnowledgeGraph context.
2. **PLAN** — LLM call: generate a step-by-step action plan.
3. **EXECUTE** — For each step: call tool via HTTP to sandbox, parse output, update KG.
4. **REFLECT** — LLM call: evaluate results, determine if sub-agents are needed.
5. **SPAWN** — Optionally spawn child agents with sub-tasks.
6. **REPORT** — Return structured findings to parent.
7. **DONE** — Agent terminates, resources released.

3.4 Context & Memory Management

- **In-context memory**: the current LLM context window, managed by a compression module that summarizes older turns when approaching token limits.
- **Persistent memory**: the KnowledgeGraph JSON, shared across all agents in a scan session.

4. Sandboxed Execution Environment

4.1 Architecture

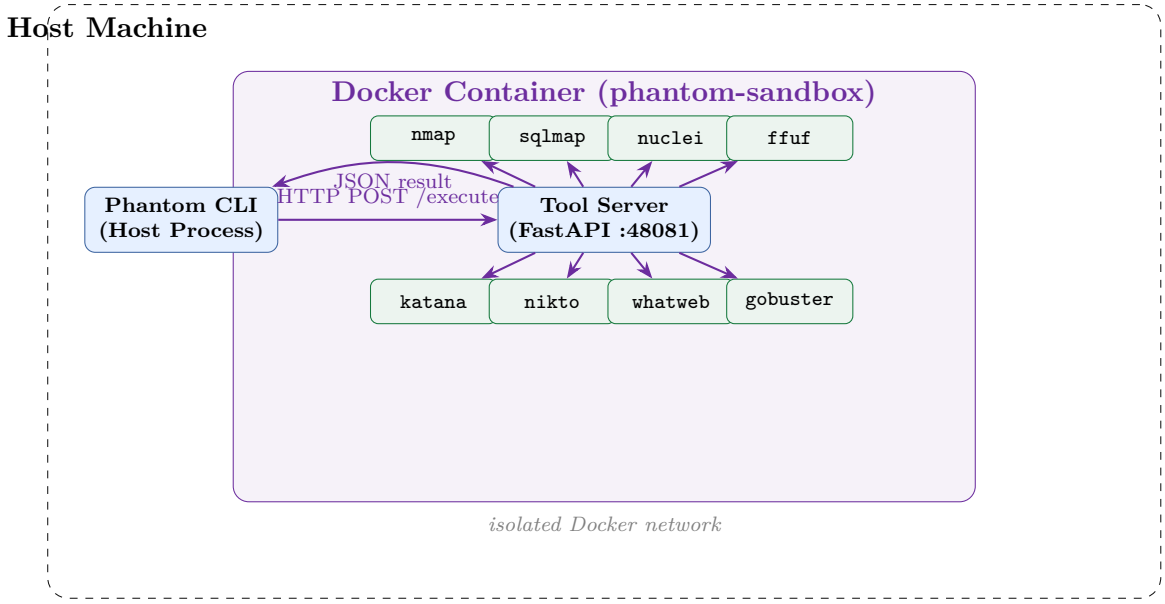

Figure 3: Sandbox architecture: the tool server inside the container mediates all tool access

4.2 Tool Server API

Listing 2: Tool Server API endpoints

```
# Health check
GET  /health
-> {"status": "healthy", "sandbox_mode": true, "active_agents": 0}

# Execute a tool
POST /execute
Authorization: Bearer <TOOL_SERVER_TOKEN>
Content-Type: application/json
{
  "agent_id":  "recon-001",
  "tool_name": "terminal_execute",
  "kwargs":    {"command": "nmap -sV -p 80,443 target.example.com", "
      timeout": 120}
}
-> {"content": "Nmap scan report ...", "exit_code": 0}
```

4.3 Security Isolation

- Tool execution runs as the `pentester` user inside the container (not root).
- The container has no persistent volumes; all data is ephemeral.

- A unique per-session bearer token is generated and injected; the server rejects unauthenticated requests (HTTP 401).
- The container runs on the default bridge network: reachable to the target, isolated from host internal services.

4.4 Pre-installed Security Tools

Table 3: Security Tools in the Phantom Sandbox (v0.8.5)

Tool	Version	Capability
nmap	7.98	Port scanner, service/version detection, OS fingerprinting
sqlmap	1.10	SQL injection detection and exploitation
nuclei	3.x	Template-based vulnerability scanning (CVEs, misconfigurations)
ffuf	2.x	Web fuzzer: directory discovery, parameter fuzzing, vhost brute-force
katana	1.x	Fast web crawler with JavaScript rendering
gobuster	3.x	Directory, DNS, and vhost brute-forcer
nikto	2.x	Web server misconfiguration scanner
whatweb	0.5.x	Web technology fingerprinter
wafw00f	2.x	WAF and CDN identification
hydra	9.x	Network authentication brute-forcer
wapiti	3.x	Black-box web vulnerability scanner
semgrep	1.x	Static code analysis for leaked/injected source
playwright	1.x	Headless browser for JavaScript-heavy app testing
curl / httpx	—	HTTP clients for manual request crafting
ipython	9.x	Python REPL for custom scripting inside sandbox

5. LLM Integration Layer

5.1 Provider-Agnostic Design

Phantom uses **LiteLLM** as a universal LLM interface. The same agent code works with any supported model; switch providers by changing one environment variable.

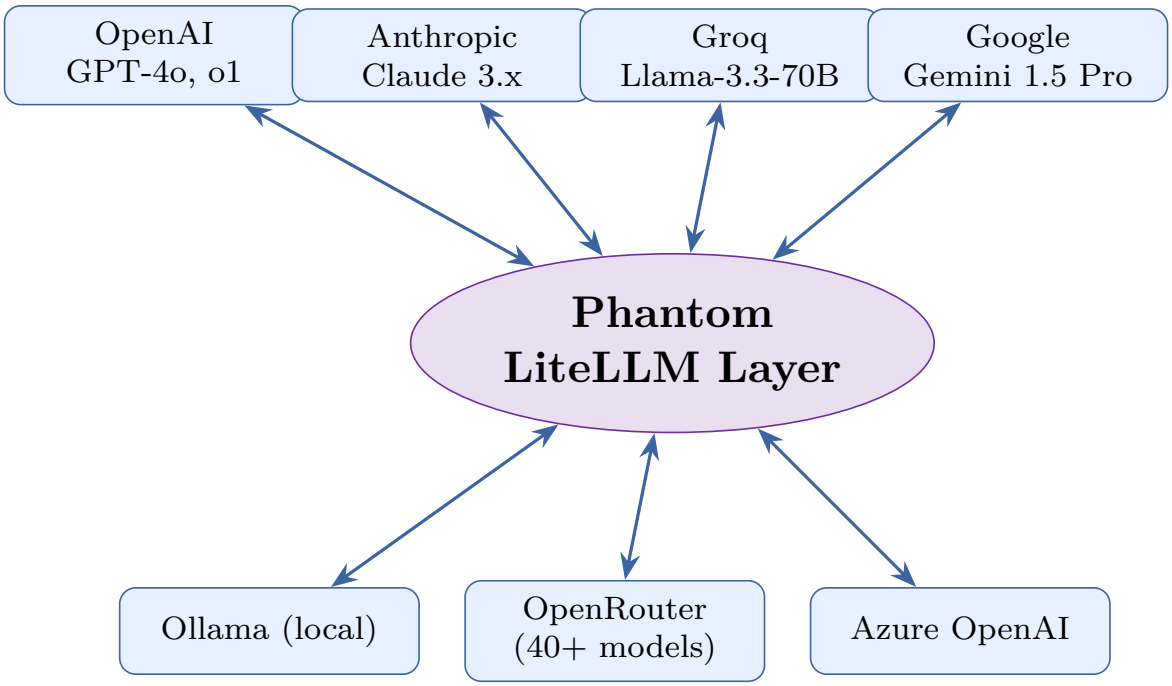

Figure 4: LiteLLM abstraction: swap LLM providers with a single environment variable

5.2 Configuration

Listing 3: Configuring an LLM provider

```
# Via environment variables (recommended for CI/CD)
export PHANTOM_LLM="openai/gpt-4o"
export LLM_API_KEY="sk-..."
phantom scan --target https://example.com

# Persistent CLI config
phantom config set llm      openai/gpt-4o
phantom config set api_key  sk-...

# Local Ollama model (no API key needed)
phantom config set llm ollama/llama3.1:8b
```

5.3 Supported Model Identifiers

Table 4: LLM Provider Identifiers

Provider	Identifier
OpenAI	`openai/gpt-4o`, `openai/o1-preview`
Anthropic	`anthropic/claude-sonnet-4-5`
Groq	`groq/llama-3.3-70b-versatile`
Google	`google/gemini-1.5-pro`
Ollama	`ollama/llama3.1:8b`, `ollama/mistral:7b`
OpenRouter	`openrouter/<any-model>`
Azure	`azure/gpt-4`

5.4 Retry & Fallback Chain

Phantom retries failed LLM calls with exponential backoff (`tenacity`). A configured fallback chain tries providers in order, ensuring scan continuity through rate limits or transient errors.

6. Scan Workflow

6.1 Scan Modes

Table 5: Scan Mode Comparison

Property	Quick	Standard	Deep
Estimated time	5–15 min	20–45 min	1–3 hours
Recon depth	Surface	Moderate	Comprehensive
Exploitation attempts	Low	Medium	Exhaustive
Sub-agent spawning	Disabled	Limited	Full tree
Directory fuzzing	Small wordlist	Medium	Large + custom
JavaScript rendering	No	Partial	Full Playwright
Post-exploitation	No	No	Yes
Differential mode	Yes	Yes	Yes

6.2 Scan Pipeline

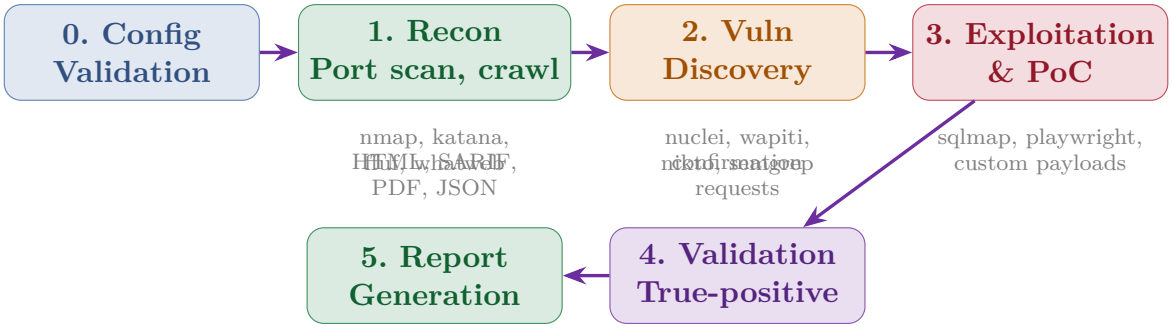

Figure 5: Sequential scan pipeline phases

6.3 Differential Scanning

When run against the same target multiple times, Phantom produces a *diff report* showing new vulnerabilities, fixed vulnerabilities, and persistent vulnerabilities. This is particularly useful for CI/CD re-test verification.

7. Detailed System Execution Plan

This section documents the complete execution flow across all system layers — from user invocation to final report delivery — with the exact code path, data transformations, and inter-component communication at every step.

7.1 Layer Stack

Table 6: Phantom System Layer Stack

Layer	Name	Location	Key Module
L0	CLI / TUI	Host	`interface/cli_app.py`
L1	Configuration	Host	`config/config.py`
L2	Runtime Manager	Host	`runtime/docker_runtime.py`
L3	Agent Orchestrator	Host	`agents/PhantomAgent/`
L4	LLM Gateway	Host	`litellm` (via `agents/base_agent.py`)
L5	Tool Server	Sandbox	`runtime/tool_server.py`
L6	Security Tools	Sandbox	nmap, sqlmap, nuclei, ffuf, ...
L7	Knowledge Graph	Host	`models/knowledge_graph.py`
L8	Report Engine	Host	`reports/`

7.2 Step-by-Step Execution Trace

7.2.1 Step 1 — CLI Invocation (L0)

```
phantom scan --target https://app.example.com \
             --scan-mode quick --non-interactive
```

Typer parses arguments, validates the target URL is reachable, displays an authorization confirmation, and in interactive mode launches the Textual TUI.

7.2.2 Step 2 — Configuration Resolution (L1)

Settings are resolved in priority order:

1. Environment variables: `PHANTOM_LLM`, `LLM_API_KEY`, `PHANTOM_IMAGE`
2. Persistent config: `~/.phantom/config.json`
3. Built-in defaults: `scan_mode=standard`, `max_iterations=300`

Pydantic v2 validates the final config; raises a descriptive error if the LLM model is not set.

7.2.3 Step 3 — Sandbox Startup (L2)

Listing 4: Container lifecycle inside docker_runtime.py

```
# 1. Check if the image exists locally; if not, docker pull
# 2. Generate a random per-session token (secrets.token_urlsafe(32))
# 3. docker run -d
#         -e TOOL_SERVER_TOKEN=<token>
#         -e TOOL_SERVER_PORT=48081
#         -p <random_host_port>:48081
#         ghcr.io/usta0x001/phantom-sandbox:latest
# 4. Poll GET /health every 2s (timeout: 60s)
# 5. Return SandboxInfo(container_id, host_port, token)
```

The token is never written to disk. It is injected as an environment variable and purged with the container.

7.2.4 Step 4 — Root Agent Initialization (L3)

```
state = EnhancedAgentState(
    agent_name    = "Phantom Root Agent",
    task          = f"Pentest: {target}",
    sandbox_id    = sandbox.container_id,
    sandbox_token = sandbox.token,
)
orchestrator = PhantomAgent(config=config, state=state)
orchestrator.run()    # Begins the agentic loop
```

7.2.5 Step 5 — Reconnaissance Phase (L3 + L4 + L5 + L6)

1. **LLM call (L4):** Root asks the LLM what to do first; LLM returns `spawn recon_agent(target)`.
2. **Recon Agent** activates and begins its own planning loop.
3. **Tool calls (L5/L6):**

```
POST /execute  {"tool_name":"terminal_execute",
                "kwargs":{"command":"nmap -sV -p 80,443,8080
                   target"}}
POST /execute  {"tool_name":"terminal_execute",
                "kwargs":{"command":"whatweb --color=never
                   target"}}
POST /execute  {"tool_name":"terminal_execute",
                "kwargs":{"command":"ffuf -u target/FUZZ -w /opt
                   /seclists/common.txt"}}
```

4. **KG update (L7):** Discovered ports, technologies, endpoints written to graph.
5. Recon Agent returns structured summary to Root Agent.

7.2.6 Step 6 — Vulnerability Analysis Phase (L3 + L4)

The **Vulnerability Analysis Agent** receives full KG context and reasons about likely vulnerability classes given the tech stack (e.g., WordPress → plugin CVEs; PHP + direct SQL → injection risk). No tools are invoked — this is pure LLM reasoning producing a prioritised `VulnerabilityCandidate` list.

7.2.7 Step 7 — Exploitation Phase (L3 + L4 + L5 + L6)

For each candidate, an **Exploitation Agent** is spawned:

Listing 5: Example exploitation calls

```
# SQL injection
POST /execute {"kwargs":{"command":"sqlmap -u 'target/login?id=1' --
   dbs --batch"}}
# CVE template scan
POST /execute {"kwargs":{"command":"nuclei -u target -t cves/ -
   severity critical,high"}}
# XSS probe
POST /execute {"kwargs":{"command":"curl -s 'target/search?q=<script>
   alert(1)</script>'"}}
```

Confirmed findings become `Vulnerability` objects with CVSS score, evidence, and PoC. In deep mode, a **Post-Exploitation Agent** may be spawned to measure impact.

7.2.8 Step 8 — Validation Phase (L3 + L4)

A **Validation Agent** re-executes the minimal reproducer for each finding. False positives are marked and excluded from the final report. Confirmed findings move to `KnowledgeGraph.verified_vulns`.

7.2.9 Step 9 — Report Generation (L8)

```
engine = ReportEngine(knowledge_graph)
engine.render_html()      # phantom_runs/<id>/report.html
engine.render_markdown() # phantom_runs/<id>/report.md
engine.render_json()      # phantom_runs/<id>/report.json
engine.render_sarif()     # phantom_runs/<id>/report.sarif
engine.render_pdf()       # phantom_runs/<id>/report.pdf
```

7.2.10 Step 10 — Sandbox Teardown (L2)

```
docker stop  phantom-sandbox-<uuid>
docker rm    phantom-sandbox-<uuid>
# Token purged; all ephemeral container data destroyed
```

7.3 Full Data Flow Diagram

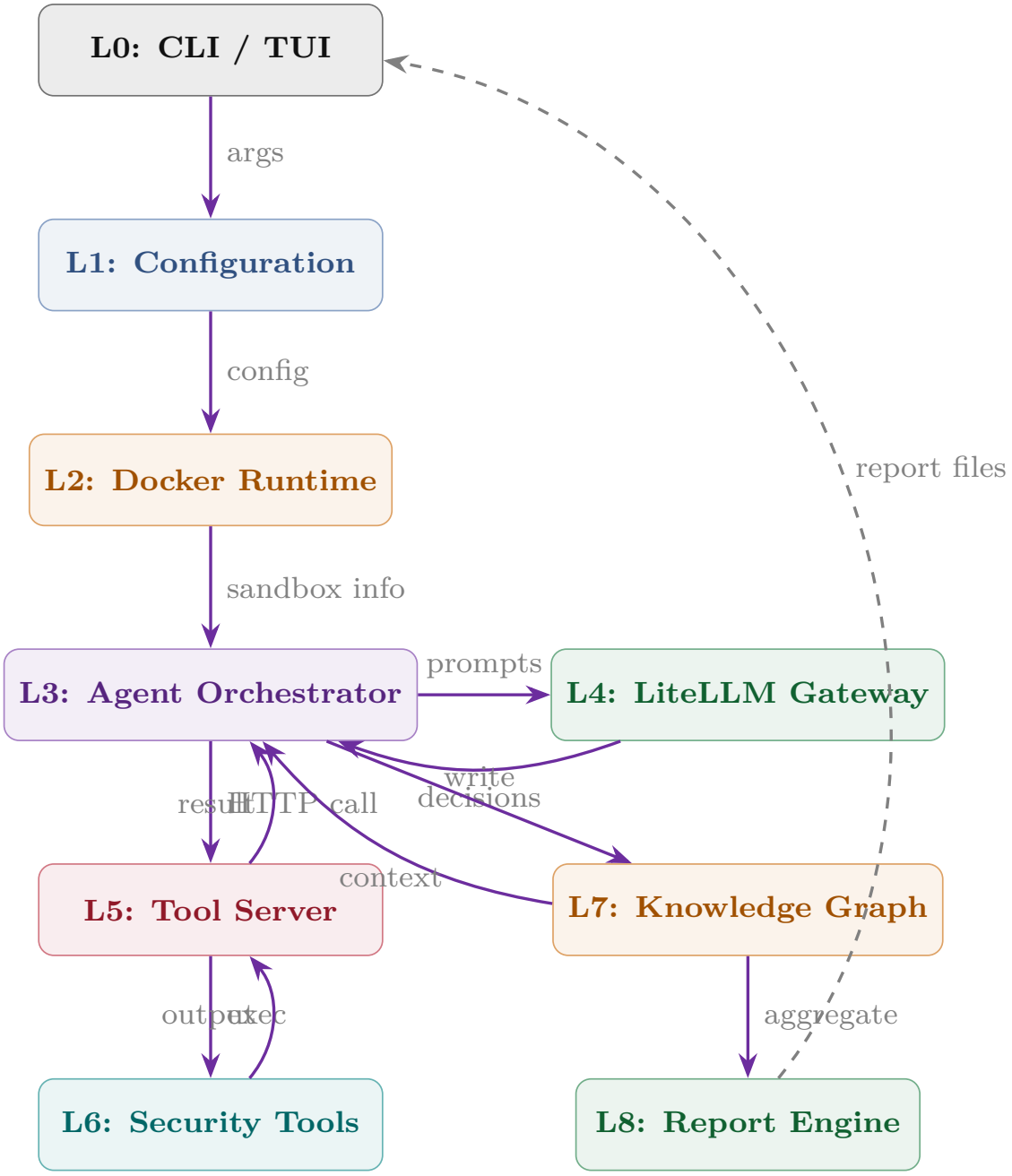

Figure 6: Complete Phantom data flow across all 9 system layers (L0–L8)

8. Knowledge Representation

8.1 Knowledge Graph

All information discovered during a scan is stored in a shared **KnowledgeGraph** updated in real time by all active agents:

- **Hosts & Services** — IPs, open ports, service banners, TLS certificates.
- **Endpoints** — URLs, HTTP methods, parameters, authentication requirements.
- **Technologies** — Frameworks, CMS, server software, version numbers.
- **Vulnerabilities** — `Vulnerability` objects with CVE, CVSS, OWASP, MITRE, evidence, PoC.
- **Attack chains** — Directed edges between chainable vulnerabilities.

8.2 Vulnerability Data Model

Listing 6: Simplified Vulnerability Pydantic model

```
class Vulnerability(BaseModel):
    id:                 str             # SHA-256 fingerprint
    title:              str
    severity:           Severity        # CRITICAL/HIGH/MEDIUM/LOW/INFO
    cvss_score:         float           # 0.0 - 10.0
    cvss_vector:        str             # CVSS v3.1 vector string
    cve_id:             Optional[str]   # CVE-YYYY-NNNN
    owasp:              str             # e.g. "A03:2021 - Injection"
    mitre_ttp:          str             # e.g. "T1190"
    endpoint:           str
    parameter:          Optional[str]
    evidence:           str             # Raw tool output
    proof_of_concept:   str             # Reproducer command/payload
    remediation:        str
    references:         List[str]
    first_seen:         datetime
    last_seen:          datetime
```

9. User Interface

9.1 Command-Line Interface

Listing 7: Phantom CLI reference

```
# Start a scan (interactive TUI)
phantom scan --target https://example.com

# Non-interactive (CI/CD)
phantom scan --target https://example.com \
             --scan-mode quick --non-interactive --output ./results/

```

```
# Configuration
phantom config show
phantom config set llm      groq/llama-3.3-70b-versatile
phantom config set api_key  <KEY>

# Reports
phantom report list
phantom report show <scan-id>
phantom report export <scan-id> --format pdf
```

9.2 Text User Interface (TUI)

Textual-based TUI (interactive mode) displays: live agent activity log, real-time findings panel, per-phase progress indicators, and keyboard shortcuts for pause/inspect/abort.

9.3 CI/CD Integration

Listing 8: GitHub Actions integration

```
- name: Run Phantom Security Scan
  run: |
    pip install phantom-agent
    phantom scan --target ${{ secrets.STAGING_URL }} \
                 --scan-mode quick --non-interactive
  env:
    PHANTOM_LLM: "openai/gpt-4o"
    LLM_API_KEY: ${{ secrets.OPENAI_KEY }}

- name: Upload SARIF to GitHub Security
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: phantom_runs/latest/report.sarif
```

10. Reporting System

Table 7: Report Output Formats

Format	File	Purpose
HTML	`report.html`	Rich formatted report with severity charts and remediation.
Markdown	`report.md`	Git-friendly; embeds in GitHub wikis, Confluence, Notion.
JSON	`report.json`	Machine-readable; integrates with ticketing tools.
SARIF	`report.sarif`	GitHub Advanced Security standard; appears in the Security tab.
PDF	`report.pdf`	Executive-ready printable report.

Every vulnerability is automatically tagged with: OWASP Top 10 2021 (A01–A10), MITRE ATT&CK TTP, CWE identifier, and CVSS v3.1 score with vector string.

11. Installation & Deployment

11.1 Prerequisites

- **Docker** — for the sandbox container.
- **Python 3.12+** — for the Phantom CLI.
- **LLM API key** — from any supported provider.

11.2 Installation

Listing 9: Install via pipx (recommended)

```
pipx install phantom-agent    # isolated env, no dependency conflicts
# or
pip install phantom-agent     # standard install

phantom --help                # verify
phantom config show           # check configuration
```

What gets installed: `pip install phantom-agent` installs the full Phantom CLI with all required Python dependencies (LiteLLM, Pydantic v2, Rich, Typer, Textual, Docker SDK, etc. — approximately 60–80 packages). Security tools (nmap, sqlmap, nuclei, etc.) are *not* installed on the host; they live exclusively inside the Docker sandbox image, which is pulled automatically on the first scan. The package registers one executable: `phantom`.

11.3 Published Images

Table 8: Published Docker Images

Image	Purpose	Size
`redwan07/phantom:latest`	Phantom CLI (Docker image)	743 MB
`ghcr.io/usta0x001/phantom-sandbox:latest`	Kali sandbox with all tools	14 GB
`ghcr.io/usta0x001/phantom-sandbox:0.8.0`	Sandbox — pinned version	14 GB

First-run note: The sandbox image (14 GB) is pulled automatically on the first scan. On slow connections, pre-pull it manually:

```
docker pull ghcr.io/usta0x001/phantom-sandbox:latest
```

12. Testing & Verification

12.1 Test Suite

Table 9: Test Results as of v0.8.5

Category	Tests	Pass Rate
Module imports	25	100%
Configuration loading/persistence	12	100%
LLM response parsing	18	100%
Vulnerability data models	20	100%
Tool argument parsing	42	100%
Scope validation	8	100%
SARIF output formatting	8	100%
Memory compression	7	100%
Agent state transitions	6	100%
Total	**146**	**100%**

```
pip install -e ".[dev]"
pytest tests/ -v --tb=short
# Result: 146 passed in ~18 seconds
```

12.2 Sandbox Verification (February 2026)

Table 10: Sandbox Verification Results

Check	Result	Detail
Container startup	✓ PASS	<30 seconds
Tool server `/health`	✓ PASS	`{"status":"healthy","sandbox_mode":true}`
Bearer auth enforcement	✓ PASS	Unauthenticated requests ⇒ HTTP 401
nmap	✓ PASS	Nmap 7.98
sqlmap	✓ PASS	sqlmap/1.10#stable
nuclei	✓ PASS	nuclei v3.x
Python/FastAPI	✓ PASS	FastAPI 0.121.0, pydantic 2.x
Tool execution	✓ PASS	Returns stdout + exit code

Published on PyPI: `pip install phantom-agent` installs the latest stable release and registers one executable: `phantom`. Run `phantom -help` to verify.

13. Security & Ethical Considerations

13.1 Authorization Enforcement

Before any scan, Phantom: (1) displays a legal warning, (2) requires explicit confirmation, (3) validates target URL and scope.

13.2 Scope Validation

Phantom enforces strict scope. Any tool call targeting an out-of-scope address is blocked before execution.

13.3 Data Privacy

Scan data is stored locally only (`phantom_runs/`). The LLM receives tool outputs and target responses; the tool server token is never included in prompts.

13.4 Responsible Use

Phantom is intended for: authorised internal security assessments, bug bounty hunters on in-scope targets, developers testing their own applications, and academic research on dedicated lab environments.

> Use against systems without written authorisation is illegal under computer fraud and abuse laws in most jurisdictions.

14. Project Status & Publication

Table 11: Phantom Publication Status (February 2026)

Channel	Address	Status
GitHub	`github.com/Usta0x001/Phantom`	✓ Live
PyPI	`pypi.org/project/phantom-agent/`	✓ v0.8.5
Docker Hub (CLI)	`docker.io/redwan07/phantom`	✓ Latest
GitHub Container Registry	`ghcr.io/usta0x001/phantom-sandbox`	✓ Latest

15. Conclusion

Phantom is a complete, production-ready autonomous penetration testing agent with the following key properties:

- **Full autonomy**: target URL to structured report with zero human intervention.
- **Real exploitation**: findings are backed by working PoC code, not theoretical assessments.
- **Clean two-component architecture**: CLI on host; all offensive tools inside an isolated Docker sandbox.
- **Provider flexibility**: LiteLLM abstraction — any LLM provider substitutable without code changes.
- **Proven correctness**: 146/146 unit tests passing; sandbox verified end-to-end in February 2026.
- **Easy distribution**: `pipx install phantom-agent` is the single install step; sandbox image auto-pulled.

Future directions: multi-target parallel scanning, authenticated application testing (cookies/CSRF), mobile and REST API scan modes, web dashboard for distributed scan management.

A. Quick Reference

Listing 10: Complete quick reference

```
# -- Install ----------------------------------------------
pipx install phantom-agent

# -- Configure LLM (pick one) -----------------------------
phantom config set llm openai/gpt-4o
phantom config set api_key sk-...
# Recommended free option:
export PHANTOM_LLM="groq/llama-3.3-70b-versatile"
export LLM_API_KEY="<GROQ_FREE_KEY>"

# -- Run a Scan -------------------------------------------
phantom scan --target https://app.example.com
phantom scan --target https://app.example.com --scan-mode quick --non
    -interactive
phantom scan --target https://app.example.com --scan-mode deep

# -- View Results -----------------------------------------
phantom report list
phantom report show <scan-id>
phantom report export <scan-id> --format pdf

# -- Pre-pull sandbox image -------------------------------
docker pull ghcr.io/usta0x001/phantom-sandbox:latest
```

B. Environment Variables Reference

Table 12: Environment Variables Accepted by Phantom

Variable	**Description**
`PHANTOM_LLM`	LiteLLM model identifier
`LLM_API_KEY`	API key for the LLM provider
`LLM_API_BASE`	Override base URL (e.g. for OpenRouter or a local proxy)
`PHANTOM_IMAGE`	Docker image override for the sandbox
`PHANTOM_DEBUG`	Enable verbose debug logging (`1` or `true`)
`PHANTOM_LLM_FALLBACK`	Comma-separated fallback model chain